

PAPER NAME

ReedMuller_22August2024.docx

AUTHOR

Manimegalai Rajkumar

WORD COUNT

4031 Words

CHARACTER COUNT

20794 Characters

PAGE COUNT

12 Pages

FILE SIZE

755.4KB

SUBMISSION DATE

Aug 22, 2024 8:54 AM GMT+5:30

REPORT DATE

Aug 22, 2024 8:55 AM GMT+5:30

● 20% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

- 16% Internet database
- 12% Publications database
- Crossref database
- Crossref Posted Content database

● Excluded from Similarity Report

- Submitted Works database
- Bibliographic material
- Quoted material
- Cited material
- Small Matches (Less than 8 words)

Implementation and Power Consumption Analysis of Reed Muller Codes: A Comparative Study of Array-Based and Booth Multipliers on FPGA

Abstract. In this present and future of computer technology, data transmission has become very crucial for every meager part of an individual's life. Data transmission in the form of texts, pictures and videos have become a part of the daily routine. It has become a mandatory part of everyone's lives as there is a need of transmission of data from one place to another no matter if it would be for short or for longer distances. Not only the distance, but also the fast transmission of data is required. Various techniques have been proposed for the fast transmission of data. But, the effect of noise or any other unwanted signal during the transmission is impossible to separate practically. Along with the fast transmission it is necessary that the data is being accurately transmitted, so as to not deliver a corrupted or a wrong data to the receiving party. Therefore, the need for data transmission with no errors through communication channels has been increased, which makes the study of detection and correction of errors significant. In this paper, various aspects and different approaches of the encoding and decoding of the Reed Muller technique, one of the most ancient and frequently used Error-correction and Error-detection techniques, has been pondered. Traditional error correction and detection techniques like hamming code and their failures are discussed. A bullseye view of the application of this technique has been studied. The methodology for the python implementation of the said technique has been proposed in this paper along with the comparison of consumption of power for the Reed Muller Encoding using Array Multipliers and Encoding using Booth Multiplier has been studied.

Keywords: Error-correction, Error-detection, Majority Logic Check, Codeword, Redundant bit, Transmitter, Receiver, Array Multiplier, Booth Multiplier.

1 Introduction

1.1 Need for Error Detection and Error Correction

With the need for faster and quick data transmission over longer distances, there comes the need for the check for the reliability of data. While an individual often looks for fast-paced data transmission, they often forget about the security and loss of data that comes along with it. Receiving the exact message as it was transmitted is equally as important. When an information is passed from a transmitter to the receiver, noise will be present in the channel. These noises are responsible for the unreliable transmission of data.

The readers may wonder, why can't we just remove the noise from the channel? What is the need for these error-correction and error-detection techniques? The answer is, it is impossible to remove noise from any media. Thereby sorting to the aforementioned techniques. To keep this prospect in check, came the concept of Coding Theory. Coding

theory, also known as algebraic coding theory, is the design of error-correcting and error-detecting for reliable transmission of information across disturbed or noisy channels.

1.2 Theory of Error Detection and Error Correction

Error Detection is the process of detecting whether the data that is received contains any error in it, so that necessary actions can be taken in order to get the actual information. It plays a crucial role in communication systems [1]. But, it does not in any way correct the faulty information. The results in both the transmitter and receiver are compared to detect errors. If they do not match, error is present.

Error-Correction is the process of correcting the errors to retrieve the correct and true message or information. Parity Checking, Checksums, Hash Functions, Frame Check Sequence (FCS) and Cyclic Redundancy Check (CRC) are some famous error detection techniques. Some of the error correction techniques are hamming codes, Turbo codes, BCH codes, Reed Muller, Reed Solomon and LDPC codes. The Reed Muller Codes are simple and easy to decode. They have the capability of correction multi-bit burst errors. The Reed Solomon Codes and the Walsh-Hadamard Codes are generalized by the Reed Muller Codes [1]. The codeword length of the Reed Muller code is very high, which requires a lot of space and because of this factor, the power dissipation will be more.

2 Literature Survey

The uniqueness of the linear code has been demonstrated by Yanling Chen and HanVick [2] in their discussion of the first order Reed Muller Code. The first order Reed Muller Code is made of linear codes that have the same length, dimension, and minimum distance between them. Additionally, it has been demonstrated that sub-codes with lengths shorter than 256 will have a shorter distance, which is equivalent to either the upper or lower range for all linear codes with comparable lengths and dimensions. Upputuri Neelima and Fazel Noorbasha [3] have proposed the encoding of an 8-bit data using different and unique Reed-Muller techniques, which includes Positive Polarity Reed-Muller (PPRM), Negative Polarity Reed-Muller (NPRM) and Fixed Polarity Reed-Muller (FPRM) analyzing these techniques with respect to cost. To make this technique even more secure, they have also incorporated the hamming code technique along with the traditional encoding and decoding of the data using the Reed-Muller technique. Hyun Woo Cho and Young Joon Song [6] have proposed Auto-Decoder that enables the short length Reed Muller decryption to achieve maximum likelihood decoding faster when compared to the Fast Hadamard Transform (FHT). It was found that this method was able to achieve 1.8 times and 125 times faster decoding for RM(1,4) and RM(2,4) than the FHT Decoding process respectively.

Tuen-Hsien Tseng and Ja-Ling Wu [7] illustrated a direct implementation of the Reed Muller algorithm for the decoding of $2(r+1)$ -layer perceptrons. The multi-layer perceptrons were constructed by the concatenation of two operations implemented by the perceptrons. They also suggested alternate approaches for the construction of multi-layer perceptrons for the solving of the Reed-Muller Codes in a more effective manner. Costas Argyrides et al [8], studied the importance of the Reed-Muller Code in the

Integrated Circuit (IC) Design. The trade-off between the area and the reliability of the IC which employs different order of the Reed-Muller Techniques have been pondered. Marzieh Hashemipour Nazari et al [9] have presented a simplified hardware architecture of the recursive projection aggregation (RPA) for the decryption or the decoding process of the Reed-Muller Codes. It helps in the reduction of the computations of the hardware up to a factor of 40%. They have presented the implementation results for $RM(6,3)$ code on Xilinx Virtex-7 FPGA which showed an advantageous throughput.

Athira.J and B.Yamnua [10] have briefed the effect of area overhead in the multiple bit upsets. The paper deals with the FPGA implementation of an area-efficient Matrix Code (MC). It was compared with the Decimal MC (DMC). The paper proved that the proposed Matrix Code method is helpful in correcting errors with less area overhead compared to the Decimal Matrix Code and Reed Muller Code and Hamming Code.

Elena Zaitseva et al [11] studied the application of multiple-valued Logic (MVL) in the field of logic design which allows gates to have two stable states. The generalized Reed Muller expression is used for the representation of the MVL function. It is also applied in the Programmable Logic Array (PLA) development.

Botang Shao and Peng Li introduced a general model for array-based approximate arithmetic computing (AAAC) aimed at minimizing processing errors. A key focus is the Error Compensating Unit (ECU), essential for optimizing AAAC circuits. They offered theoretical insights for optimal error compensation which resulted in designs that can balance energy, area and accuracy [11]. S.S. Mahant-Shetti, et. al addressed power dissipation in digital multipliers, especially in the architecture of the array with introducing a novel array connection pattern. It reduces spurious transition activity, which enhances power and throughput efficiency. The study achieved up to 50% delay reduction and 35% power savings compared to conventional designs [12].

Zhijun Huang and M.D. Ercegovic introduced a high-performance, low-power linear array multiplier design using signal flow optimization in a [3:2] adder array, left-to-right leapfrog (LRLF) signal flow, and splitting the array into upper/lower parts (ULLRLF). Compared with tree multipliers, ULLRLF multipliers show similar power, delay and area for sizes up to 32 bits, offering a competitive alternative in deep submicron VLSI technology due to their regularity and shorter interconnects [13].

Shiann-Rong Kuang and Jiun-Ping Wang presented a power-efficient, configurable Booth Multiplier with dynamic-range detection and truncation, achieving significant power savings while maintaining acceptable output quality across real-life applications [14]. Kyung-Ju Cho, Kwang-Chul Lee, et.al introduced an error compensation method for a modified Booth fixed-width multiplier, using Booth encoder outputs to reduce quantization error by up to 50%.

The method applies targeted compensation to divided truncated bits, achieving up to 35% reductions in area and power consumption while maintaining similar hardware overhead [15]. Suganthi V., Hyuk Jae L., et. al, discussed that the approximate computing reduces power consumption in multimedia and deep learning applications. Radix-4 Booth Multipliers, using novel approximations in partial product generation and accumulation, offer significant area (41%) and power (49%) reductions while maintaining performance in tasks like JPEG compression [16].

Fang Yuan, Nan Ye, et. al proposed coding and decoding algorithms to enhance relay satellite wireless transmission. Experimental results demonstrate minimal error, indicating the system's practical applicability for increasing satellite transmission rates [17]. R.W. Yeung and Zhen Zhang addressed a source coding

system with multiple sources, encoders, and decoders, each with varied connectivity and reconstruction requirements. By deriving inner and outer bounds in terms of fundamental entropy regions, the paper offers a general framework for evaluation admissible coding rates and introduces new tools for multiuser information theory [18].

3 Reed Muller Code

Reed Muller Codes is one of the old and famous codes. D. E. Muller and I. S. Reed created this phenomenal idea in 1954. Reed Muller Code, by itself constitutes a large number of family of codes. Smaller Reed Muller Codes are capable of forming larger Reed Muller Codes which makes its definition recursive in nature [4].

The Reed Muller Codes involve the idea of monomials and vector spaces and their relation to some particular rings of polynomials. It also requires the knowledge of Scalar Addition; Vector complement and Vector Multiplication.

The r^{th} order Reed-Muller code is denoted by $RM(r, m)$. It is defined as the set of all polynomials of degree ' r ' (at most) in the ring R_m . For the given integers m and r with $0 \leq r \leq m$, there exists a binary r^{th} order RM code, with the specifications as follows:

Code length of Reed Muller Code is given by equation 1.

$$n = 2^m \quad \dots\dots\dots(1)$$

Dimension is given by equation 2.

$$k(r, m) = 1 + {}^mC_1 + {}^mC_2 + \dots\dots + {}^mC_r \quad \dots\dots\dots(2)$$

Minimum distance between tow codes is given by equation 3.

$$d_{min} = 2^{m-r} \quad \dots\dots\dots(3)$$

Error Correcting Capability is given by equation 4.

$$(2^{m-r}-1)/2 \quad \dots\dots\dots(4)$$

For $RM(r, m)$, the generator matrix is given by equation 5.

Consider the mapping: $\psi : R_m \rightarrow F_2^{2^m}$

$$\psi(0) = 00 \dots\dots 0 \text{ (for } 2^m \text{)}$$

$$\psi(1) = 11 \dots\dots 1 \text{ (for } 2^m \text{)}$$

$$\psi(x_0) = 11 \dots\dots 1 00 \dots\dots 0 \text{ (for } 2^{m-1} \text{ respectively)}$$

$$\psi(x_1) = 11 \dots\dots 1 00 \dots\dots 0 11 \dots\dots 1 00 \dots\dots 0 \text{ (for } 2^{m-2} \text{ respectively)}$$

$$\psi(x_2) = 11 \dots\dots 1 00 \dots\dots 0 11 \dots\dots 1 00 \dots\dots 0 11 \dots\dots 1 00 \dots\dots 0 11 \dots\dots 1 00 \dots\dots 0 \text{ (for } 2^m \text{)}$$

$$G_{RM(r,m)} = \begin{bmatrix} \Psi(1) \\ \Psi(x_0) \\ \Psi(x_1) \\ \vdots \\ \Psi(x_{m-1}) \\ \Psi(x_0x_1) \\ \Psi(x_0x_2) \\ \vdots \\ \Psi(x_{m-1}x_{m-2}) \\ \Psi(x_0x_1x_2) \\ \vdots \\ \Psi(x_{m-r}x_{m-r+1} \dots x_{m-1}) \end{bmatrix} \dots\dots\dots(5)$$

The generator matrix of $RM(2,3)$ is given by equation 6.

$$G_{RM(2,3)} = \begin{bmatrix} \Psi(1) \\ \Psi(x_0) \\ \Psi(x_1) \\ \Psi(x_2) \\ \Psi(x_0x_1) \\ \Psi(x_0x_2) \\ \Psi(x_1x_2) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \dots\dots\dots(6)$$

3.1 Encoding

Encoding is simply done using the basic LBC technique of multiplying the message with the generator matrix [4]. Example: If the message to be encoded, given by equation 8 is:

$$Msg = 01101001010 \dots\dots\dots(7)$$

Then encoded codeword is given by equation 9.

$$C = Msg * G_{RM(2,4)} = 101011111111010 \dots\dots\dots(8)$$

3.2 Decoding

Decoding of Reed-Muller, is done using one of the easy techniques called the Majority Logic Decoding. The idea of Majority Logic Decoding, is to find out whether the particular row was used to form the considered codeword that corresponds to our message through majority voting [4]. The process of decoding starts with the examination of the rows which contains the monomials of degree r . The characteristic vectors of the

row is calculated one by one and the dot product of each of these vectors with the received codeword is taken and recorded. If 1 is the resultant for majority of the dot products that were calculated, then it is finalized that the row was in fact used in the construction of the original codeword and it is given its position in the vector given for the original message which corresponds to the row 1. Similarly, if the resultant of the majority comes out to be 0, then we consider that this row was not taken for the formation of the original codeword. Which will make the corresponding part in the vector to be 0. This technique is applied for each and every row that corresponding to the monomials with the degree 'r'. After all the rows are checked, the resultant vector is then proceeded to be multiplied with the m choose r rows of the generator matrix. This results into s and s is added to the message that was received by the receiver to get the original transmitted message [4].

Example: Message to be decoded is $v = 0\ 0110110$ in $RM(2,3)$
 Row x_1x_2 has characteristic vectors $x_0, \bar{x}_0$. The multiplication with v is given by equations 10 and 11.

$$\begin{aligned} v \cdot \Psi(x_0) &= 0 & \dots\dots\dots(10) \\ v \cdot \overline{\Psi(x_0)} &= 0 & \dots\dots\dots(11) \end{aligned}$$

Now the message becomes, $Msg = \text{-----}0$
 Similarly, row x_1x_2 has characteristic vectors $x_0, \bar{x}_0$. Multiplication with v is given by equations 12 and 13.

$$\begin{aligned} v \cdot \Psi(x_1) &= 1 & \dots\dots\dots(12) \\ v \cdot \overline{\Psi(x_1)} &= 1 & \dots\dots\dots(13) \end{aligned}$$

1 gets added to the message, $Msg = \text{-----}1\ 0$
 The quantity 's' computed and is given in equation 14.

$$s = [0\ 1\ 0] \cdot \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} = [1\ 0\ 1\ 0\ 0\ 0\ 0] \quad \dots\dots\dots(14)$$

On adding s to v we will get $1\ 0\ 0\ 1\ 0\ 1\ 1\ 0$. Further, the degree of the monomial is reduced by 1, i.e. $r-1$ and then the same process is followed. The characteristic vectors of the row x_2 is given by: $x_0x_1, x_0\bar{x}_1, \bar{x}_0x_1$. The multiplication with vector u is given by the equations 15 to 18.

$$\begin{aligned} u \cdot (\Psi(x_0) * \Psi(x_1)) &= 1 & \dots\dots\dots(15) \\ u \cdot (\overline{\Psi(x_0)} * \Psi(x_1)) &= 1 & \dots\dots\dots(16) \\ u \cdot (\Psi(x_0) * \overline{\Psi(x_1)}) &= 1 & \dots\dots\dots(17) \\ u \cdot (\overline{\Psi(x_1)} * \overline{\Psi(x_1)}) &= 1 & \dots\dots\dots(18) \end{aligned}$$

Now, $Msg = \text{---}1\ 0\ 1\ 0$

On continuing this procedure, we will get the original message without errors as:
 $0\ 1\ 1\ 1\ 0\ 1\ 0$

3.3 The First Order Reed Muller Code

It is denoted as $R(1,m)$ and it consists of all vectors $u_0 1 + \sum_{i=1}^m u_i v_i = 0 \text{ or } 1$, corresponding to the linear Boolean functions. The first order RM Code with length 8 is defined as a polynomial of the first degree is given by equation 19.

$$a_0 1 + a_1 v_1 + a_2 v_2 + a_3 v_3, a_i = 0 \text{ or } 1 \quad \dots\dots\dots(19)$$

The encoding and decoding process of the first order RM Code is easy and simple. The efficiency of error-correction is also high. For instance, the National Aeronautics and Space Administration (NASA) utilized the first order Reed Muller Code $RM(1,5)$ for the transmitting pictures from Mars to Earth. The first order Reed Muller Code, $RM(1,5)$ can decode without any problem if there are at most 7 errors without any problem[1]. There is a appreciatively high error correction capacity for the code length $n = 32$. This leads to the ability to withstand more noisy channels for the $RM(1,5)$. This property makes it very suitable for the Mariner 9 spacecraft for transmitting pictures from Mars to Earth [1]. Reed Muller Codes often lead to high power dissipation, increasing costs and compromising data security. To address these issues, alternative encoding techniques such as Positive Polarity Reed-Muller (PPRM), Negative Polarity Reed-Muller (NPRM), and Fixed Polarity Reed-Muller (FPRM) are used [3].

Consider an 8-bit data encoded using PPRM, NPRM, and FPRM techniques for secure transmission. This encoded data is further encoded with Hamming Code to ensure error-free transmission. Given the noisy channel, redundant bits are used to detect and correct errors using Hamming Code. The error-free data is then decoded with the respective Reed-Muller decoding techniques to retrieve the original 8-bit data. A basic block diagram illustrating this process is provided in Fig. 1[3]

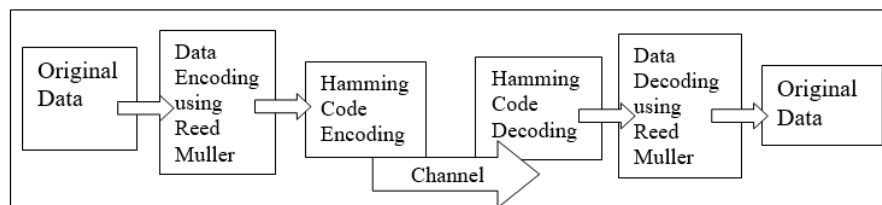


Fig. 1. Block diagram of the Transmitter and the Receiver

4 Python Implementation

The Python Implementation of the Reed Muller Code is done with the help of various inbuilt and user defined class and functions and modules.

The first step of the program starts with the user entering the values for 'r' and 'm'. If $m < r$, then the program terminates, else it enters to the further main part of the program. We create an object *rm* with correspondence to the class reed muller by passing 'r' and 'm' as the arguments to it. The strength of the RM code, message length, codeword length, generator matrix and error vectors are displayed if required. The message to be

encoded is obtained from the user and further encoded by passing it to the encode function and the codeword is displayed. The received codeword, with or without errors, is got as input from the user and it is decoded by passing it to the decode function, which gives the corrected codeword by using the Majority Logic Decoding and displayed. Table 1 shows the various functions used for the implementation.

Table 1. Functions used in the program with their use

Name of the Function	Function Description
<i>Binom(n, k)</i>	Returns binomial coefficient n choose k
<i>vector(m, i)</i>	Returns a binary vector with 1 at position i , and 0 elsewhere
<i>vect_mult(*vecs)</i>	Returns element-wise multiplication of given vectors
<i>vect_add(*vecs)</i>	Returns element-wise addition of given vectors
<i>vect_comp(x)</i>	Returns element-wise complement of the input vector x
<i>vect_mod(x, modulo)</i>	Returns element-wise modulo operation of the input vector x
<i>dot_pro(x, y)</i>	Returns dot product of vector x and y .
<i>rows(m, S)</i>	If S is empty, returns a list containing of single vector with all 1's, else return element-wise multiplication of xi_row with each row in S_rest rows.
<i>char_vector(n, S)</i>	Returns a binary vector of length n with 1's at positions specified in S , and 0 elsewhere
<i>strength():</i>	Returns the strength of the message.
<i>message_length():</i>	Returns the length of the message.
<i>block_length():</i>	Returns the length of the codeword.
<i>construct_matrix()</i>	Returns the constructed generator matrix
<i>encode(word)</i>	Returns the encoded codeword
<i>decode(eword)</i>	Returns the decoded original message without errors.

4. 1. Outputs

In Fig.2. the value for r and m has been given and we get the no. of errors that could be detected and the possible length of the codeword is given as output. We also feed the input message or the data that has to be transmitted and we are given with the calculated codeword as the output. Fig. 3. Shows the corrupted codeword. When we transmit the modulated codeword from the transmitter side onto the channel, the noise present in the channel will corrupt the codeword.

Fig. 3. shows the error-corrected message. In the receiver side, the Reed-Muller Technique is applied to correct the errors and thus receive the correct and original data.

```

Enter r:1
Enter m:4
No. of errors that can be corrected: 3
Length of the message to be encoded: 5
Length of the codeword: 16
Message bit:
10101
Encoded Codeword:
1010010110100101

```

Fig. 2. Encoding of the given message

```

Received Codeword:
1011011010100101
Decoded Message:
10101

```

Fig. 3. Corrupted codeword and Decoded Message without any Errors.

5 Hardware Implementation of Encoder

5.1. Encoding using Array Multiplier

Implementation is first done using a normal array multiplier which is used to multiply the generator matrix and the input bit array.

Array multiplier is constructed using multiple adders and AND-gates. It generates all the partial products after only one AND-gate delay. Then, it sums up all partial products sequentially. The advantage of this structure is that the arrangement of its adders is very regular and is favorable for layout. It also can be realized with parallel structure. However, it occupies more area and hardware than that of iterative multiplier. Consider two binary numbers A and B, of m and n bits. There are m n summands that are produced in parallel by a set of m n AND gates. n x n multiplier requires n (n-2) full adders, n half-adders and n2 AND gates. In Array Multiplier worst case delay would be (2n+1) td. The results below are obtained when the design is coded using Verilog in Xilinx.



Fig. 4. Encoded Message for RM(1,4) using array multiplier

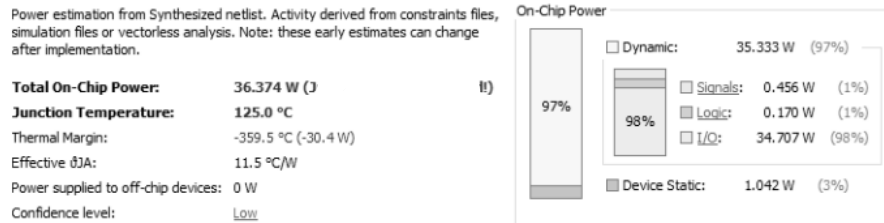


Fig. 5. Power report of RM(1,4) encoder using array multiplier

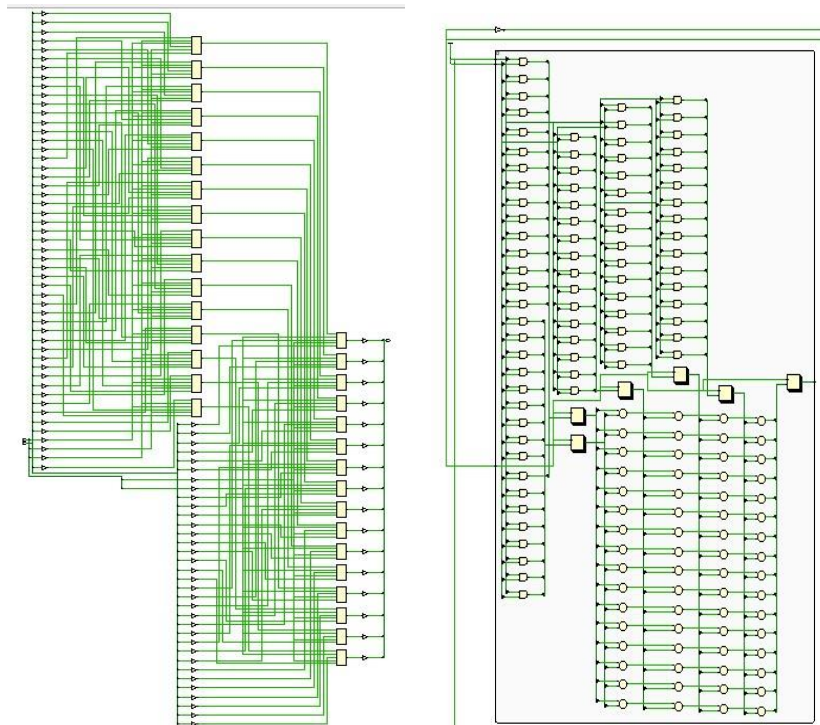


Fig. 6. Schematic of RM(1,4) using array multiplier

5.2. Proposed design: Encoding using Booth Multiplier

The encoder is designed using a Booth Multiplier. The function of the encoder is to create a codeword by multiplying the input data array with its corresponding generator matrix. The generator matrix is made using a set of vectors corresponding to the RM code. The main ideology behind using the Booth multiplier is that the Booth's algorithm cuts the number of required partial products in half. This increases the speed by reducing the total number of partial product sums that must take place.

The Booth algorithm gives a procedure for multiplying binary integers in signed 2's complement representation in efficient way, i.e., a reduced number of additions/subtractions required. It operates on the fact that strings of 0's in the multiplier require no addition but just shifting and a string of 1's in the multiplier from bit weight 2^k to weight 2^m can be treated as $2^{(k+1)}$ to 2^m . The Booth algorithm requires examination of the multiplier bits and shifting of the partial product in all the multiplication schemes. Before shifting, the multiplicand may be added to the partial product, subtracted from the partial product, or left unchanged according to following rules:

1. The multiplicand is subtracted from the partial product when the first least significant 1 in a string of 1's is encountered in the multiplier.
2. The multiplicand and the partial product are added when the first 0 (provided that there was a previous '1') in a string of 0's is encountered in the multiplier.
3. The partial product does not change when the multiplier bit is identical to the previous multiplier bit.

Booth's algorithm is widely used in hardware implementations of multiplication operations, including digital signal processors, microprocessors, and FPGAs.

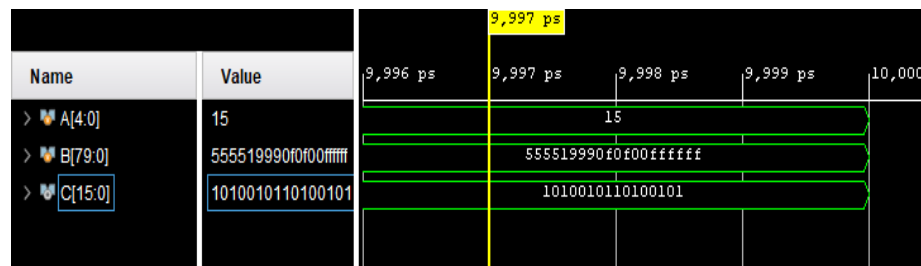


Fig. 7. Encoded Message for RM(1,4) using Booth multiplier

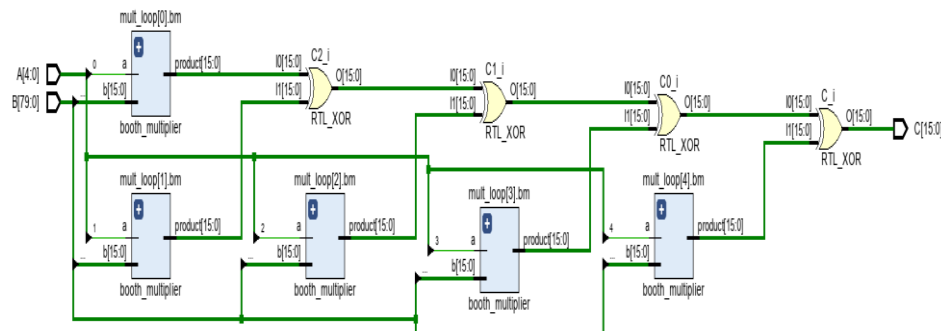


Fig. 8. Schematic of RM(1,4) using Booth multiplier

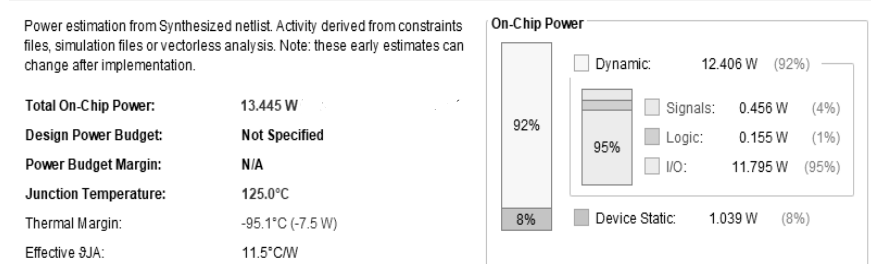


Fig. 9. Power report of RM(1,4) using Booth multiplier

6 Results

The Table 2 shows the comparison of the power consumed by array multiplier-based encoder and the booth multiplier based encoder.

Table 2. Power comparison between Array Multiplier and Booth Multiplier			
RM(1,4) using Array Multiplier		RM(1,4) using Booth Multiplier	
Total On Chip Power: 36.374 W		Total On Chip Power: 13.445 W	
Dynamic	Device Static	Dynamic	Device Dynamic
35.333 W (97%)	1.042 W (3%)	12.406 W (92 %)	1.039 W (8%)

It is observed that switching from Array Multiplier to Booth Multiplier reduces the Total On Chip Power by 63.04 %.

7 Conclusions

In this paper, the need for Error-Detection and Error-Correction in communication systems is studied. The importance of the coding theory along with the need for the error correcting capability of multi-bit burst errors are discussed. Various papers which provided versatile ideas about the Reed-Muller Code and how Reed Muller codes find use in satellite communication and ground stations for the transmission of high-rate data and cryptographic applications for secure data transmission are also discussed. The encoding and decoding process of Reed Muller Code along with its advantages, disadvantages and applications are briefed. The process of Reed Muller Code in Satellite Communication and the secure data transmission is highlighted. The proposed methodology of Python Implementation along with the error-correction results are obtained. The Encoding of Reed Muller Codes based on Array Multipliers and Booth Multipliers were implemented and the power consumption in both the methods are compared. It is found that using Booth Multipliers could reduce the Total On Chip Power by 63.04%. Future work could explore integrating AI-driven ECC design with Reed Muller Codes, optimizing them with reinforcement learning and genetic algorithms [19]. This approach may enhance performance beyond traditional methods and adapt to evolving requirements in diverse applications.

● 20% Overall Similarity

Top sources found in the following databases:

- 16% Internet database
- 12% Publications database
- Crossref database
- Crossref Posted Content database

TOP SOURCES

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	core.ac.uk Internet	3%
2	epfarms.org Internet	3%
3	interscience.in Internet	3%
4	enggjournals.com Internet	1%
5	rroj.com Internet	1%
6	arxiv.org Internet	<1%
7	researchgate.net Internet	<1%
8	amrita.edu Internet	<1%
9	Dave K. Kythe, Prem K. Kythe. "Algebraic and Stochastic Coding Theor... Publication	<1%

10	ftp.cs.toronto.edu	Internet	<1%
11	Zhijun Huang, M.D. Ercegovac. "High-performance low-power left-to-ri...	Crossref	<1%
12	ethesis.nitrkl.ac.in	Internet	<1%
13	scholars.lib.ntu.edu.tw	Internet	<1%
14	Kenneth H. Rosen, Douglas R. Shier, Wayne Goddard. "Handbook of Dis...	Publication	<1%
15	Shu Lin. "Linear Block Codes", The Kluwer International Series in Engin...	Crossref	<1%
16	Hyun Woo Cho, Young Joon Song. "High Speed Decoding for High-Rate...	Crossref	<1%
17	D. Jackuline Moni, P. Eben Sophia. "Design of low power and high spee...	Crossref	<1%
18	Heng You, Yong Hei, Jia Yuan, Weidi Tang, Xu Bai, Shushan Qiao. "Desi...	Crossref	<1%
19	S. Mitra, Y. Hayashi. "Neuro-fuzzy rule generation: survey in soft comp...	Crossref	<1%
20	A. Canteaut. "Decomposing bent functions", IEEE Transactions on Infor...	Crossref	<1%
21	D. R. Hankerson, D. G. Hoffman, D. A. Leonard, C. C. Lindner, K. T. Phel...	Publication	<1%

22	Peter Beelen, Gregor Leander. "A new construction of highly nonlinear ..."	<1%
	Crossref	
23	ia801504.us.archive.org	<1%
	Internet	
24	keysight.com	<1%
	Internet	
25	Subir Kumar Sarkar, Asish Kumar De, Souvik Sarkar. "Foundation of Di..."	<1%
	Publication	
26	Svetlana N. Yanushkevich, Vlad P. Shmerko, Sergey Edward Lyshevski....	<1%
	Publication	
27	YUEN-HSIEN TSENG, JA-LING WU. "Decoding Reed-Muller codes by m..."	<1%
	Crossref	
28	library.fridoverweij.com	<1%
	Internet	
29	pqc-hqc.org	<1%
	Internet	
30	vardhaman.org	<1%
	Internet	